Adityaa Saha

Final Year Undergraduate
Department of Computer Science and Engineering
Indian Institute of Technology Bhubaneswar

23cs01002@iitbbs.ac.in
Adityaa Saha
Adityaa-Saha17
ats17 whocares17

Work Experience

IIT Bhubaneswar, School of Electrical Sciences *May'26 – Jul'26*
Research Intern | Prof. Devashree Tripathy

- Worked on architecture-aware **Sparse Matrix Multiplication (SpMM)**, optimizing global, shared, and distributed memory access patterns on **NVIDIA T4 GPUs** using **CUDA**
- Replaced **2:4 structured sparsity** with a **block-sparse (BCSR) Tensor Core** path after identifying that T4's Turing architecture lacks the Ampere-only `mma.sp.sync` instruction
- Descoped DSM-style multi-GPU execution to framework-level partitioning via **PyTorch/NCCL** under Colab single-T4 and Kaggle dual-T4 (no `nvcc`) constraints
- Built CUDA **benchmarking** scripts evaluating memory optimization strategies across shared and global memory hierarchies

Projects

TCP/IP Stack *Personal Project*

- Built a **userspace TCP/IP stack** in **C++23** from a raw **TAP device**: Ethernet, ARP, IPv4/ICMP, UDP, and a full **RFC 9293 TCP state machine** on a single-threaded **epoll** loop
- Implemented three-way handshake, four-way close, retransmission with **Karn/Jacobson RTT estimation**, and **Reno congestion control** (slow start, congestion avoidance, fast retransmit/recovery)
- Designed around `std::expected` error handling, bounds-checked cursors over `std::span`, and **zero-copy** fixed buffers on the hot path
- Validated with a toy **HTTP/1.0 server** over the custom stack, tested via `tcpdump`, `arping`, and `curl`

HTTP Load Balancer *Personal Project*

- Built an **L4/L7 load balancer** from scratch in C++ on Linux with **edge-triggered non-blocking** epoll I/O handling **C10K**-scale concurrency
- Implemented **weighted round robin**, **least connections**, and **power-of-two-choices**, plus **consistent hashing** with virtual nodes for cache-aware routing
- Designed active/passive **health checking** with hysteresis and slow-start recovery, and a **circuit breaker** (closed/open/half-open) to isolate failing backends
- Built an **HTTP/1.1 parser** with state-machine request handling, keep-alive pooling, and **request smuggling** (CL.TE) defenses
- Added hot-reloadable **TOML** config via atomic pointer swap, structured **JSON logging**, and **Prometheus**-style RED metrics

Key–Value Storage Engine (LSM Tree) *Personal Project*

- Built a **log-structured merge-tree** store in C++ with a **skip-list memtable**, **write-ahead log** for crash recovery, and immutable **SSTables** with block indexes and **Bloom filters**
- Designed compaction behind a **pluggable strategy interface** with **leveled** and **size-tiered** policies to compare write vs. read amplification
- Decoupled background flush/compaction from the write path using **atomic manifest swaps** so readers see consistent snapshots without blocking

Recursive DNS Resolver *Personal Project*

- Built a **multithreaded DNS server** in C++ with a hand-rolled **wire codec** (name compression, **EDNS0/OPT**, TC-bit truncation) over UDP and TCP on a fixed-size **thread pool**
- Implemented an **authoritative zone engine** (A, AAAA, CNAME, MX, NS, TXT, SOA) with wildcards, CNAME chasing, SOA-driven negative answers, and a conditional forwarder
- Designed a **shared-mutex LRU cache** with TTL decay and negative caching, plus a per-IP **token-bucket rate limiter** to bound amplification abuse
- Added blocklisting, a **Prometheus** `/metrics` endpoint with latency histogram, rotating logs, and a `dig`/`nc` suite for malformed-packet resilience

32-bit RISC-V CPU Simulator *Personal Project*

- Implemented a **cycle-accurate RV32I** simulator with a **5-stage pipeline** (IF, ID, EX, MEM, WB) and a **two-pass assembler** resolving labels and forward references
- Added a **hazard detection unit** with EX/MEM and MEM/WB **forwarding**, load-use interlock stalls, and control-hazard flushing on taken branches
- Instrumented with **branch prediction**, per-stage tracing, and **CPI/stall/flush counters** exported as JSON

Technical Skills

Languages: C, C++ (11/17/23), Go, Python, CUDA, Bash
Systems & Networking: Linux systems programming, epoll & event-driven I/O, sockets, TCP/IP, DNS, HTTP/1.1, multithreading, atomics & lock-free concurrency, storage engines, GPU memory hierarchies
Tools & Libraries: Git, GDB, CMake, PyTorch, NCCL, Prometheus, tcpdump, dig, curl, LaTeX

Relevant Courses *(* Ongoing)*

Data Structures	Operating Systems	Advanced Algorithms*
Discrete Structures	Database Systems	Artificial Intelligence*
Design & Analysis of Algorithms	Computer Networks	Networks & System Security*
Computer Organization & Architecture	Formal Language & Automata Theory	Math. Foundations of CS*